

Incident Triage — Engine & Platform Design

The complete design, in one place: the system, **the engine on LangGraph**, the capability layer, the full data model, the physical/LLD detail, operations, and the release schedule. Domain-neutral; incident triage is the first domain.

Infrastructure Resilience & Operations · v1.0 · combines the HLD + LLD + data model + delivery · companion: the Presentation (leadership) · worked on INC-4821 / payments-api

A · THE SYSTEM

A1 · What & why — goals

A2 · Three levels — domain-neutral

A3 · Architecture

B · THE ENGINE ON LANGGRAPH

B1 · Loading the playbook → compiling the run

B2 · The run state

B3 · The phase loop — plan → execute

B4 · Running the capability layer in the loop

B5 · The gate, checkpoint, resume

B6 · Phase transitions — conditional edges

B7 · A full run trace (INC-4821)

B8 · The live session — multi-user, stateless

B9 · The investigation graph — runtime & tools

C · CAPABILITY LAYER & GOVERNANCE

C1 · Connection priority

C2 · The registry

C3 · govern()

C4 · Intent resolution

C5 · Autonomy ladder & feedback

D · THE DATA MODEL

D1 · The investigation graph

D2 · The phase record

D3 · The four phase outputs

D4 · Subject & storage planes

D5 · Feedback · D6 · Incident document

E · PHYSICAL / LLD

E1 · Postgres DDL

E2 · Mongo collections

E3 · Engine API

E4 · Errors, retries, recovery

F · OPERATIONS

F1 · The console

F2 · Cross-cutting

G · DELIVERY

G1 · Milestones & timeline

G2 · NFRs

A · The system

A1 What & why — goals

A governed, in-house **investigation engine** for production operations. One engine runs a versioned **playbook** through gated, audited phases; it builds a **graph** of the situation, reasons over it with the model, calls **governed capabilities** (never tools directly), and a human

approves every write. The first domain is incident triage; the same engine runs other domains by loading a different playbook.

Goal	How the design meets it
No vendor lock-in	the engine calls abstract capabilities; a source can change without touching the engine (Part C).
No domain lock-in	everything domain-specific is in the playbook (types · phases · outputs); the engine, shapes, registry, governance, storage and console are universal (A2).
Governed at the boundary	every call passes <code>allow / ask / deny</code> ; every write is human-approved; closing is human (B5, C3).
Durable + reversible	checkpointed, exactly-once, resumable; every write carries a rollback (B5, E4).
Evidence-based + auditable	every fact carries an <code>evidence_ref</code> ; the Step journey is the audit trail (D, F2).
Gradual	Amelia runs alongside, wrapped as a governed capability, dialled down per proven slice (G1).

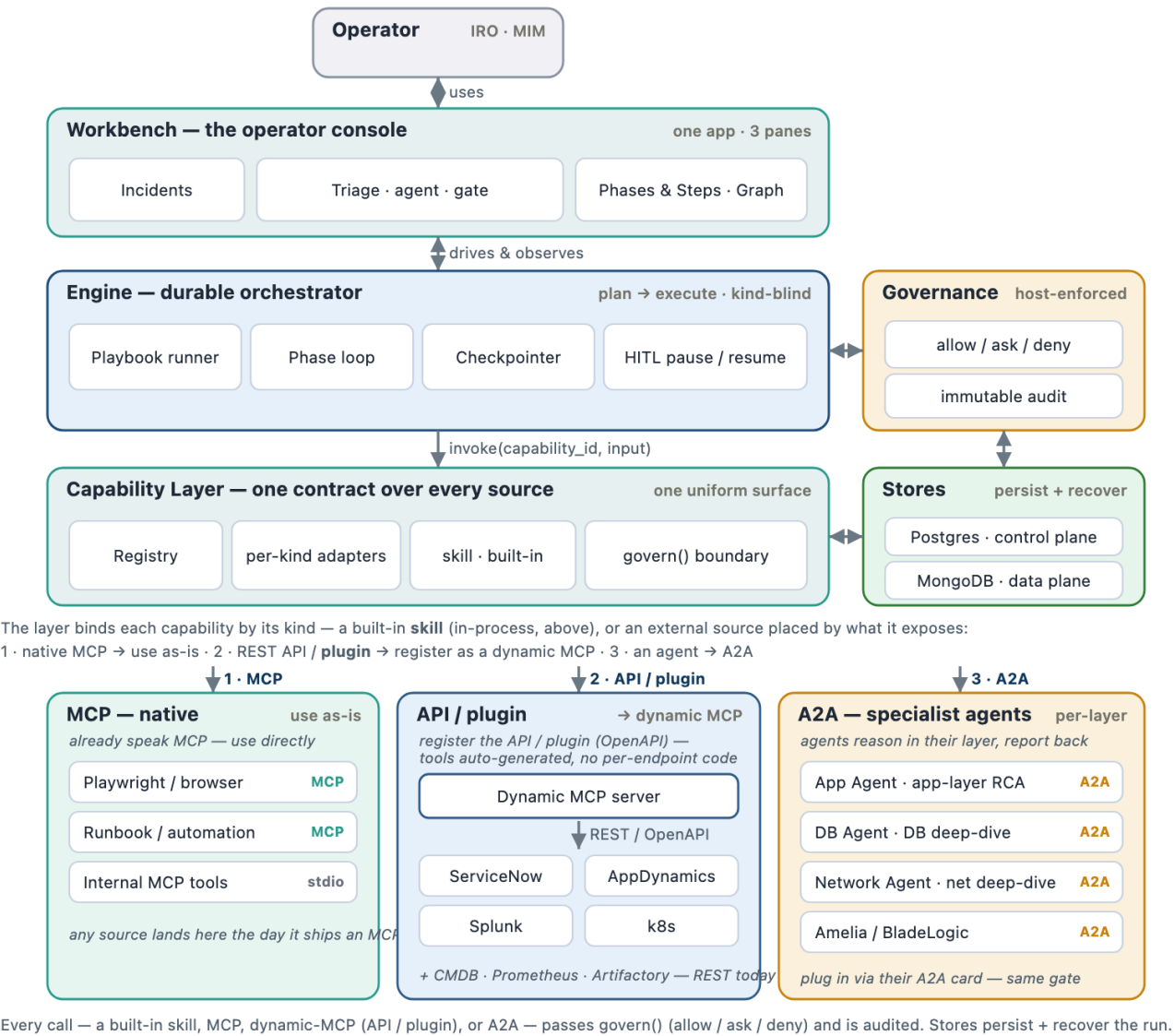
A2 Three levels — domain-neutral

Level	What it is	Domain-specific?
Engine	the shapes (Node·Edge·Fact·PhaseRecord·Step), the run loop, <code>govern()</code> , gates, retries, clustering, the registry, feedback, storage, the console	No — universal
Playbook	the type system (<code>graph_schema</code>), the phases (<code>+needs / effect</code>), the output contracts	Yes — one file per domain
Runtime	the concrete nodes/edges/records of one investigation	instances

The subject is named domain-neutrally as a `SubjectRef {domain, id, kind}` (unique key (domain, id)), never a hardcoded `incident_id` — so the engine never assumes the word "incident". incident-triage fills it `{app-incident, INC-4821}` ; provisioning `{provisioning, REQ-123}` .

A3 Architecture

System architecture — components, sub-components, and how every source connects



Every call — a built-in skill, MCP, dynamic-MCP (API / plugin), or A2A — passes govern() (allow / ask / deny) and is audited. Stores persist + recover the run.

Component	Responsibility	Where
Engine (orchestrator)	loads the playbook; runs the phases; drives the per-phase model loop; enforces effect ; checkpoints; re-enters	Part B
Capability layer	resolves intents → concrete capabilities; binds sources by kind; runs govern()	Part C
Capability registry	Provider · DeclaredCapability · CapabilityPolicy	C2 · E1
Graph builder	builds/updates the per-incident graph; clustering; the cause walk	D1
Stores	Mongo (incident docs) · Postgres (playbook + registry) · LangGraph checkpointer (run state)	D4 · E1–E2
Console	the single operator UI — three panes + inline gate + graph view	F1

Connectors

native MCP · dynamic-MCP (OpenAPI) · A2A specialists
(App/DB/Network; Amelia/BladeLogic wrapped)

C1

B · The engine on LangGraph

This is the heart of the design. The engine is a **LangGraph** application. It (B1) *loads a playbook and compiles it into a state machine*, (B3) runs each phase as a *model-driven agent loop* that (B4) *calls the capability layer in the loop* and folds results into the graph, (B5) *pauses at the gate and checkpoints*, and (B6) *moves between phases by conditional edges* — looping for confidence and backtracking on failed recovery. B7 is a full trace.

B1 Loading the playbook → compiling the run

On startup (and on every playbook publish), the engine **loads the playbook row** from Postgres and **compiles it into a LangGraph StateGraph**. The playbook is data; the compiled graph is what runs.

1. **Parse + validate** the front-matter: `phases[]` (each with `id · effect · needs · output · min_confidence?`), `graph_schema` (`node_types`, `edges`, `facts`, `labels`, `conventions`), `output_schemas`, `defaults`, `unknown_access`, `error_handler`.
2. **Register the domain types** from `graph_schema` (so the graph builder + UI know the vocabulary) and the `output_schemas` (JSON Schema, used to validate each phase output, D3/E).
3. **Build the state machine**: one node per phase, edges in declared order, conditional edges for loop/backtrack (B6), `interrupt_before` on writes (B5), and the checkpointer.

```
def compile_run(playbook):
    # playbook = the Postgres row, pinned @version
    pb = parse_frontmatter(playbook.body_md, playbook.phases, playbook.schemas)
    validate(pb)  # phases, graph_schema, output_schemas well-formed
    # — COMPILER-TIME INVARIANTS (fail at LOAD, not mid-run) —
    # 1. every write phase MUST set gate_writes=true (else invoke mid-phase would see
    #     access=ask but no interrupt_before to pause → Denied exception uncaught)
    for p in pb.phases:
        if p.effect == "write" and not p.gate_writes:
            raise ValueError(f"write phase {p.id} must set gate_writes=true")
    # 2. every phase.output name must be a registered output type (else mid-run validation fails)
    for p in pb.phases:
        if p.output not in OUTPUT_TYPES:
            raise ValueError(f"phase {p.id}: unknown output type {p.output}")

    register_types(pb.graph_schema)  # node_types / edges / facts → graph builder + UI
    register_schemas(pb.output_schemas)  # JSON Schema per phase output
    layer.unknown_access = Access(playbook.unknown_access)  # reconcile playbook policy into layer

    g = StateGraph(RunState)
    for ph in pb.phases:
        g.add_node(ph.id, make_phase_node(ph, pb))  # B3 — the agent loop for this phase
```

```

g.set_entry_point(pb.phases[0].id)                # assess
wire_edges(g, pb.phases)                          # sequential + conditional (B6)
gate_writes_phases = [p.id for p in pb.phases if p.gate_writes]
return g.compile(checkpointer=PostgresSaver(...), # B5 – durable, resumable
                 interrupt_before=gate_writes_phases or None) # pause before each gate_writes pha

```

Because the phases, types and contracts all come from the playbook, **compiling a different playbook yields a different engine behaviour with zero code change** — the domain-neutral promise, realized at compile time.

B2 The run state

One run = one `LangGraph thread_id`. The state is the working set every phase reads and writes; `LangGraph` checkpoints it after each step.

```

class RunState(TypedDict):
    subject:      SubjectRef          # {domain, id, kind} – the thing under investigation
    playbook_ref: str                 # "incident-triage@1.0.0" (pinned for reproducibility)
    graph:        InvestigationGraph  # nodes / edges / facts – SHARED across all phases (Part D)
    phase_records: Annotated[list, append] # one PhaseRecord per phase entered (D2)
    messages:     Annotated[list, append] # the conversation (operator + agent)
    current_phase: str
    pending_decision: dict            # {decision, actor} – carried into resumed gate phase; consu
    status:       str                 # running|waiting_approval|waiting_input|done|failed – route

```

The **graph** is the shared world (it persists and grows across phases); each **PhaseRecord** is one phase's journey + typed output. On phase completion the engine **writes through** to the Mongo incident document (D6); the live state stays in the checkpointer until close.

B3 The phase loop — plan → execute

Each phase node is a **model-driven agent loop**, bounded by the phase's `needs` (which intents it may use) and `effect` (read-only or write). The model **plans**, then loops — pick a need, call a capability, fold the result into the graph, log a Step, update the output — until the typed output is **sufficient** (its required fields are present + validate).

```

def make_phase_node(ph, pb):
    def node(state):
        rec = PhaseRecord(id=f"{state.subject.id}:{ph.id}:{attempt(state, ph)}",
                          phase=ph.id, goal=ph.goal, plan=model.plan(state, ph), steps=[])
        while not sufficient(rec.output, pb.output_schemas[ph.output]):
            need, args = model.next_action(state, rec, allowed=ph.needs) # 1 · model picks an INTENT
            result      = capability_layer.call(need, args, effect=ph.effect, state=state) # 2 · Part B4 /
            fold_into_graph(state.graph, result) # 3 · nodes / facts / edges +
            rec.steps.append(Step(kind="tool_call", capability=result.cap_id,
                                input=args, result=result.data,
                                touched=result.node_ids, evidence=result.evidence))
            rec.output = model.update_output(rec, state.graph) # 4 · build the typed output i
            if model.wants_operator(rec): return pause(state, "waiting_input") # ask a question
        validate(rec.output, pb.output_schemas[ph.output]) # 5 · contract check (E)

```

```

state.phase_records.append(rec); write_through_to_mongo(state)
return state
return node

```

Steps are emergent, not scripted — the planner (model) decides the next need each iteration given the current graph state and phase output. The loop ends when `sufficient(output, output_schema)` is true (all required fields present + validates). The output is refreshed after each tool call via `planner.update_output()`. Steps record tool calls (`kind=tool_call`), reasoning (`kind=reasoning`), gate decisions (`kind=decision` , recorded FIRST on re-entry if approved), and other events — the Step list is the audit trail.

B4 Running the capability layer in the loop

Step 2 above — `capability_layer.call(need, ...)` — is where intents become governed, real calls, **every iteration of the loop**. The layer never lets the model touch a tool directly.

```

def call(need, args, effect, state):
    caps = resolve_intent(need, effect)          # C4 - match DeclaredCapabilities by intent + effect
    cap = rank(caps)[0]                         # by trust, freshness, fit
    dec = govern(cap, playbook_of(state))        # C3 - effect x access (allow | ask | deny)
    if dec.access == "deny": raise Denied(cap)    # model must find another path / escalate
    if dec.access == "ask": return request_gate(cap, args, state) # B5 - pause for approval
    key = input.get("idempotency_key") if isinstance(input, dict) else None
    if key is not None and key in cache: return cache[key] # exactly-once on WRITE
    adapter = adapter_for(cap.provider)         # bind by provider id
    if adapter.kind != provider(cap).kind: raise Denied(f"adapter kind mismatch") # C1 binds by kind
    raw = adapter.invoke(cap.id, input)         # native MCP | dynamic-MCP | A2A dispatcher
    if key is not None and effect == Effect.write: cache[key] = raw # cache ONLY writes
    return Result(cap_id=cap.id, data=raw,
                  node_ids=raw.touched,        # which graph nodes this read/wrote
                  evidence=raw.evidence_refs)  # deep links - fold into Fact.evidence_ref + output

```

In the loop	What happens
resolve_intent	need (e.g. metrics) → DeclaredCapabilities tagged with that intent whose effect is compatible with the phase's effect. The filter uses the authoritative effect — <code>policy.effect ?? effect_hint ?? unknown</code> , the SAME value <code>govern()</code> enforces — not the raw <code>effect_hint</code> . So a capability whose hint says <code>read</code> but whose policy corrects it to <code>write</code> is still excluded from a read-only phase: a read phase provably never even ranks a write/unknown cap (FR12/AC1).
govern	Calls <code>govern(cap, playbook)</code> : resolves <code>cap.effect_hint × CapabilityPolicy.access</code> at call time. <code>read</code> → usually allow (invoke, audit); <code>write</code> → ask (pause the run, wait for operator decision); <code>unknown</code> effect → <code>playbook.unknown_access</code> policy (allow ask deny per policy, reconciled into layer at compile B1).
invoke	If <code>access=ask</code> and not yet approved: raise <code>NeedsApproval</code> (pause, no invoke); if <code>deny</code> : raise <code>Denied</code> ; else dispatch. <code>invoke()</code> is the exactly-once chokepoint (FR5): a write carrying an <code>idempotency_key</code> is dispatched at most once — a retry or crash-replay with the same key returns the cached result, never double-applies. The

	bound adapter's <i>kind</i> must match its provider's declared kind (native MCP · dynamic-MCP-wrapped API · A2A specialist agent), else <code>Denied</code> ; it returns the raw result with <code>evidence_refs</code> .
fold	Each capability result is mapped to graph mutations (nodes/facts/edges) by a per-capability fold-adapter registered in <code>FoldRegistry</code> . The adapter is the ONLY place a source's shape meets graph schema. Fold updates the in-memory graph and records touched node ids + evidence refs in the <code>Step</code> .

B5 The gate, checkpoint, resume

The gate. At compile, every `gate_writes` phase (effect=write, gate_writes=true) is named in `interrupt_before` . When the run enters such a phase, the LangGraph interrupt pauses BEFORE the phase node executes, leaving `next=[phase_id]` visible to the API (`status=waiting_approval`). The engine returns the phase spec + output so far; the operator **approves | refines | denies** via the API.

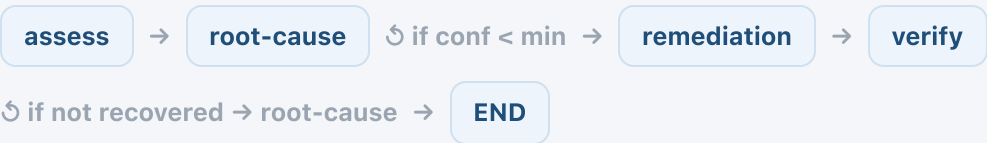
Resume path. The operator's decision {decision, actor} is posted to the API, which calls `engine.resume(thread_id, decision)` . The engine:

1. Updates the checkpoint with `pending_decision: {decision, actor}` (idempotent on decision content + idempotency_key).
2. Invokes the LangGraph thread, which re-enters the paused phase.
3. The phase node's FIRST step records `Step(kind=decision, decision=..., actor=...)` before any tool calls — the most accountability-critical event (D2).
4. Then the phase proceeds with the approved writes; `pending_decision` is consumed/nulled after the `Step` is appended.
5. LangGraph checkpoints after every step (durable + resumable).

Exactly-once. The decision is idempotent — a double-approve on the same gate with the same `decision_id` re-runs deterministically (same inputs → same result). Crash mid-phase = resume from the last checkpoint; transient failures within the phase retry per `defaults.retry` .

B6 Phase transitions — conditional edges

Phases run in declared order, but the edges are driven by each phase's own **metadata** — **never hardcoded phase ids**. This is what makes the run **adaptive and re-enterable**, and what keeps the engine domain-neutral: a differently-named playbook (different phases, different loop/backtrack points) compiles to the right graph with zero code change.



A phase that declares...	...compiles to this edge (the contract)
<code>effect: write</code> <code>+ gate_writes: true</code>	Pauses before phase executes. The phase id is added to <code>interrupt_before</code> at compile. When the run reaches this node, LangGraph halts with <code>next=[phase_id]</code> visible to the API. On resume with the operator's decision, the phase re-enters, records the decision Step (D2 audit: who approved + when), then proceeds with tool calls. Every write phase MUST declare <code>gate_writes=true</code> — validated at compile; otherwise writes would trigger <code>access=ask</code> mid-node, uncaught.
<code>min_confidence: x</code> (a loop phase)	Re-enter the same phase while <code>top_confidence(state) < x</code> ; else advance. Re-entries are capped at 5 attempts — once hit, advances anyway (inconclusive evidence proceeds; next phase may escalate) rather than infinite recursion. A phase with <code>min_confidence: null</code> never loops.
<code>backtrack_to: <earlier-phase></code> (a backtrack / terminal phase)	If the phase's <code>recovered</code> output field is true → END (close); else re-enter the named earlier phase to gather more evidence. <code>backtrack_to</code> must reference a phase that appears <i>earlier</i> in the phases list — validated at Playbook.model_validator (compile load time) , so a forward or missing target fails immediately, never silently mid-run.
neither	Advance sequentially to the next declared phase in the phases list; if last, route to END.

Halt on pause / failure. The `route_phase()` function (the compiled conditional-edge router) checks `status` first: if `status ∈ {waiting_input, failed}` it returns `halt → END`, surfacing that status to the API. Only if status is `running` does routing proceed to loop/backtrack/advance logic. This guard ensures a paused or failed phase never silently advances. The router returns exactly one token — `halt · loop · advance · backtrack · close` — which LangGraph maps to a destination node (END, self, next, or `backtrack_to`).

A re-entered phase opens a **new attempt** (`...: root-cause: 2`); the graph and prior records are retained as context, so the second pass is better-informed. A human performs the final close after the terminal phase reaches END.

Related incidents — human-guided, one focus. A run investigates **one** incident in its own focus. On launch (and as it looks out during its cycle), the engine **checks for an existing session on a similar incident**; if it finds one it **asks the operator** — link this as *related* (into a `cluster` via `member_of`), or open a new triage. It never auto-merges and never runs "at cluster scope" — each incident keeps its own session, phases, and closure; a linked cluster just lets the shared cause/fix propagate.

B7 A full run trace — INC-4821

The state machine, walked end to end on the worked incident (failed disk → RAID rebuild → DB I/O → checkout latency).

Load. Compile `incident-triage@1.0.0`; `thread_id = INC-4821:run:1`; seed the graph — `INC-4821(incident)`, `alert:7741`, from `incident-source`.

assess (read-only loop): `topology` → `app+DB+storage neighbourhood`; `telemetry` → `health`; `change-history` → `rev47` (recorded, not yet ruled out); `similar-incidents` → `INC-4820` (its `resolution` → a suggestion). → **AssessResult** (`perf · P1 · suspected_locus=DB · suggestion=failover`). Checkpoint. Edge → `root_cause`.

root-cause (read-only loop): `traces` → 75% is the DB call; `metrics` → DB I/O 28ms; `metrics (storage)` → **pay-vol disk reconstructing since 13:58**; rule out `rev47/gemini/network`. → **RootCauseResult** (`stor:pay-vol 0.9`, basis stated, path `app→db→storage`). `top_confidence 0.9 ≥ 0.7` → edge to remediation. Graph: `stor:pay-vol` gains `suspect` + a `suspected_cause` edge.

remediation (write, gated): model proposes `failover db:payments-ora → standby` with `expected_effect/blast/rollback/temporary`. `govern()` → write → **ask** → `interrupt_before` **pauses** (`status=waiting_approval`). Operator **approves** → resume → invoke (`idempotency-guarded`) → **RemediationResult** (applied; I/O 28ms→4ms) + a `followups` escalation to replace the disk.

verify (read-only loop): `telemetry + synthetic-replay` → p99 4.2s→260ms; `recovered(s)=true` → edge to END. → **VerifyResult** (`resolution = failover + disk replacement; temporary action scheduled to revert`). **Human closes**. Final state written through to the Mongo incident document (D6).

B8 The live session — shared, multi-user, on stateless servers

Two layers, not one. The **record** (graph + phase records + outputs) is the durable *audit + display* — the Mongo read-model (Part D). The **live chat session** (the conversation + the in-flight run) is a *runtime* the data model deliberately left to "LangGraph owns it" (D4). This section designs that runtime. The rule that keeps it simple: **state lives in the stores; the session is a thin layer = a lock + a polled event log.**

B8.1 · Lifecycle — free chat → shared session

free chat identity + subject → related-incident check → create or join → shared session

State	What it is · cost
Free chat	per-user, "new chat" — exploratory; no run, no graph, no playbook ; a TTL'd scratch. Near-zero cost; may never promote.
Promotion	once identity + subject {domain, id} are known → the related-incident check (above) → the operator creates a new session or joins an existing one for that subject. Human-confirmed.
Shared session	keyed by the subject = one thread = one run (B2); several operators/devices live in it at once; closes with the incident.

B8.2 · Problem 1 — the live run on stateless, load-balanced servers

One incident = one run, but requests land on any server and operators share it. The crux: a step that is **in-flight** (the agent mid-thought, streaming, calling a tool) lives in one server's memory — it isn't checkpointed yet.

Mechanism	How
Serialize the run	advancing a session takes a per-session lock — a lease with heartbeat (a Postgres advisory lock on the subject id in production, or in-memory in-process for mocks). The holder is the transient run-owner and drives the agent loop one step at a time. No second run can start for the incident. The lease has a 30s TTL ; a slow tool keeps it by calling <code>heartbeat(session_id, token, ttl=30)</code> ; a dead owner's lease expires and is stolen by another server, which resumes from the checkpoint.
Input during an in-flight step	a 2nd operator's message is queued (a per-session input queue), drained after the current step or at the next gate — merged into the one run, never spawning a second.
Reads & chat go anywhere	only advances need the lock; reads (render graph/phases) hit the Mongo read-model and chat is polled from the event log (B8.3) — both from any server. No sticky load-balancing.
Crash mid-step	the lock lease expires → another server resumes from the last checkpoint ; the lost step re-runs (reads idempotent, writes idempotency-keyed). No work lost, no double-write.

In-memory = only the active step, on the owner, transiently. The durable truth is the LangGraph checkpoint + the read-model — so any server takes over between steps.

B8.3 · Problem 2 — syncing the chat across users

Several operators, possibly on different servers, must see the same conversation, the agent's live stream, and the gate — in real time.

Mechanism	How
One event log per session	an append-only log keyed by the subject . Every event — a user message, an agent message, a gate prompt, a gate decision, a phase/graph delta — is appended with a per-session sequence number (starts at 0, increments per append, returned to the caller). The log is queryable by <code>since(session_id, after_seq)</code> — returns all events with <code>seq > after_seq</code> (exclusive lower bound); <code>snapshot_seq(session_id)</code> returns the highest seq so far (the checkpoint for join-mid-flight).
Append, don't push	the agent appends its output to the log; any connected client (on any server) sees it on its next poll — not just the operator who triggered the step.
Clients = snapshot + polled deltas	a client renders the heavy state from the read-model (the graph/phases snapshot) and applies the deltas it polls from the log. The log carries the conversation + change events; the read-model carries the structured truth.

Join mid-flight /
reconnect

fetch the **snapshot** (read-model + recent chat) and **poll from the last seq** → no lost or duplicated messages.

Backend: an **append-only event log** in the read-model store (Mongo, or Postgres); clients **poll since(seq)** for correctness, and may upgrade to **SSE** (/stream , Last-Event-ID ; B8.5) for live delivery over the same seq interface. **No push bus** — no Redis, no WebSocket fan-out: every server answers a poll/stream statelessly from the shared store, so there is **no cross-server fan-out**. (A WebSocket + a bus like Redis / Postgres LISTEN/NOTIFY remains a later option behind the same seq interface; correctness never depends on it.)

One ordered stream. Chat, graph deltas, and phase deltas all ride the *same* log under one sequence; the read-model snapshot is consistent to a seq , so a client renders the snapshot and applies only the deltas after it — no cross-path races between "the graph changed" and "the agent said...". **Scaling:** serialize *within* a session (one run-owner), run *across* sessions in parallel (different owners) — the system scales by incident count, not by serializing everything.

B8.4 • Edge cases

Case	Handling
Gate while in-flight	the write gate pauses the run (no active step) → the run-owner lock releases → the gate is answered-once (first terminal decision wins, idempotent on the result): a concurrent answer sees it already resolved and returns the same decision without re-checking writer status; the first answer itself is guarded by writer-only + terminal-only (approve deny; refine is non-terminal and excluded) — → resume re-acquires the lock.
Two operators approve at once	answered-once + optimistic checkpoint version — first wins, the rest see it resolved.
Operator joins mid-investigation	snapshot (read-model + recent chat) + start polling from that seq → current state + live.
Network drop / reconnect	snapshot + resume-from-seq; client message ids dedupe a resend.
Backtrack (Verify → Root cause)	same session/thread, a new attempt (B6); chat + graph continue — the session does not restart.
Long session / compaction	the agent's context = the current phase + prior phase OUTPUTS + on-demand graph slices (B3/B4), not the raw history. The chat <i>pane</i> shows full history from the store; the agent <i>context</i> stays bounded.
Free chat never promoted	TTL-expires; no durable cost.
Any domain	identical machinery — the session is keyed by {domain, id} , so provisioning/capacity sessions work the same.
Create-or-join race	two operators promote to the same subject at once → create_or_join() is idempotent on the subject id (a unique key) and checks authorization BEFORE

	any mutation: an unauthorized actor raises <code>NotAuthorized</code> before being recorded in <code>members</code> or <code>pen_holder</code> , so they cannot block the session. The first authorized creator makes the thread and holds the pen; the rest attach as viewers. Never two threads for one incident.
Promotion carries context	the free chat's prior messages seed the session's conversation — promotion is not a reset.
Headless / idle session	the run can advance (an autonomous step) or sit at a gate with no client connected ; events persist → a late joiner replays them; a gate that sits too long escalates (<code>error_handler</code>).
Slow vs crashed owner	the lock is a lease with a heartbeat — a slow tool keeps the lease by calling <code>heartbeat(session_id, token, ttl)</code> on each step; a dead owner lets the 30s lease expire → another server calls <code>acquire(session_id, owner, ttl=30)</code> , succeeds (the expired lease is free), and resumes from the checkpoint. Distinguishing the two is why the heartbeat is required: a running owner refreshes every step, a crashed owner does not.
Authorization mid-session	incident membership is re-checked per event (via <code>is_member(actor)</code> on every <code>post_event()</code>) and also on every read (<code>incident/events/stream/poll</code>) — a revoke drops the operator immediately on the next attempt, losing reads + writes (AC9).
A missed / failed poll	polling is liveness-only — a dropped request just retries on the next tick; the run still advances (the lock + checkpointer don't depend on it). Correctness lives in the stores, never the transport.

The whole session, in one line: a **lock** serializes the one run, a **polled event log** syncs the many clients, and the durable stores hold the truth — so any server serves any request, and the chat is shared and live without push infra, a fourth store, or sticky routing.

B8.5 · Roles, the pen, and the chat UI (MVP)

The pen. One member holds **the pen** (the single human writer) at a time — only the pen-holder may send a message or approve a gate; everyone else is a read-only **viewer**. (The pen is the *human* write-token — who may act; it sits above the **run-owner lock** of B8.2, which serialises the engine's one run — different mechanisms, same "one at a time" spirit.) The pen is handed over (`take-pen` / `release-pen` , one holder at a time; the creator starts with it). **Pen liveness:** a holder's activity (message / gate decision) refreshes a heartbeat; an idle holder past **PEN_TTL (60s)** can be force-taken by any other member via `take-pen()` , so a disconnected operator never permanently blocks writes and gate approvals. The agent runs **one unit of work to completion, then takes the next message** — strictly sequential. Approval is **single** — the pen-holder's OK is enough; **no dual approval** (a maker-checker rule is future). Roles are enforced **in the UI and re-checked at the server** on every write.

The chat UI. The chat is a heterogeneous stream of typed events ordered by `seq` ; each renders via a **widget registry** (`kind` → `component` : `text` · `agent step` / `tool-call` · `table` · `image` · `graph` · **sandboxed-iframe HTML** for untrusted/agent-generated UI). A new widget type = one renderer, no core change; untrusted HTML is isolated in a sandboxed iframe (the safety seam — aligns with the open **MCP-Apps** pattern). Transport: **SSE** (`GET /stream` , resume via `Last-Event-ID`) for the live app, polling as the simple fallback — both over the one event log, so a widget is just a richer event payload, independent of the transport.

B9 The investigation graph — runtime, tools & library

The engine owns the graph; the LLM acts on it through tools. The graph is *state* — it lives as an in-memory working model (the engine's), persisted to the stores, **never** as text the LLM holds. The model **reasons over a rendered slice** and **builds via governed tools**. (Chosen over LLM-held text — fragile, no audit, can't be shared — and over a shared graph DB — heavy, concurrent-write contention; a per-incident graph, reused across incidents via the learning loop, gives determinism + audit + bounded context.)

B9.1 · Representation & library

A per-incident, in-process graph: **nodes by id** (facts embedded), **typed edges** (out + in adjacency), per Part D. It is small (the working slice, not the estate) — so an in-memory structure is enough; **no graph DB**.

Choice	Stance
Library	networkx (Python, in-process) — mature, cycle-safe traversal (<code>descendants</code> , <code>shortest_path</code> , <code>subgraph</code>). A custom typed adjacency structure is the fallback only if footprint/perf demands; per-incident the graph is small, so networkx is the KISS default.
Where it lives	the hot working copy on the run-owner (B8.2) — loaded from the durable graph when a step starts, mutated, checkpointed. Not a persistent in-memory server.
Durable form	the Mongo incident document (the graph as JSON, D6) + the LangGraph checkpoint. The in-memory copy is transient.

B9.2 · The tool surface — how the LLM acts on the graph

The LLM never receives the raw graph; it gets a small, **governed** tool surface (capabilities, gated like any other, Part C). Mostly **read** (traverse/expand) + one **write** (annotate a finding, with provenance). Bulk graph-building is automatic — the engine **folds** each tool result (B9.4); the LLM only queries + records explicit findings.

Tool	Effect · what it does
graph.get(id)	<code>read</code> · a node + its facts. Unknown id → <code>{id, status: "unknown"}</code> ; never invents.

graph.neighbours(id, edge?, dir?)	read · adjacent nodes — expand the frontier . Optional edge filter by edge type; dir = "in" "out" (default). Caps at expand_cap=12 per node; returns {id, dir, neighbours:[stub...], more: N} where more is the overflow count (omitted if 0).
graph.walk(from, edges[], dir, until?)	read · traverse a path (e.g. depends_on / hosted_on downward to the cause). With until (a predicate dict), BFS the edge-type-filtered subgraph for the NEAREST matching node, returning {from, path, nodes:[stub...], reached: true false}. Without until, a greedy single-path frontier walk, returning reached: null. Max depth 12; cycle-safe.
graph.find(predicate)	read · nodes matching a structured predicate. Recognized keys: label · layer · kind · type · id · name · unhealthy · impacted; an unrecognized key → match fails (never silent match-all). Empty predicate {} matches all. Returns {predicate, matches:[stub...], count}.
graph.blast_radius(id)	read · the impacted set (walk affects / depends_on).
graph.path(from, to)	read · the shortest causal path.
graph.annotate(target, key, value, evidence_ref)	write · the agent records a finding (a fact or label like suspect) — validated + provenance-stamped (which step). Requires a non-empty evidence_ref — a finding without evidence is rejected with ValueError. Target must exist (fold/add it first). Returns {target, label fact, evidence_ref, by_step}. This is "build as it goes," audited not gated.

Read tools resolve effect: read-only (allowed); annotate is a write but **low-risk** — it changes the agent's own working graph, not the world — so it is **audited, not gated** like a remediation.

B9.3 · Rendering — the slice the LLM sees

Each turn a deterministic **render-slice** projects the graph into the context — bounded regardless of size:

- **In full:** the subject node, the current cause path, flagged nodes (suspect /impacted) + their facts, and the **1-hop frontier** (adjacent un-expanded nodes, as stubs the agent can expand). Neighbours are capped per-node at expand_cap=12 and the frontier is **globally** capped at frontier_cap=20; overflow neighbours fold into the collapsed count, so the invariant full + frontier + collapsed = total always holds (the slice stays ~10–30 nodes regardless of graph size).
- **Collapsed:** healthy / ruled-out / resolved → a **count + one-line summary** ("135 healthy · collapsed").

So even a 147-node incident renders ~10–30 nodes; the graph carries the memory, the slice shows only what's live. **This is the compaction** (B3) — the agent sees the distilled world +

prior phase outputs, never the raw history.

B9.4 · Lifecycle, the fold-adapter & persistence

Step	What happens
Load	rebuild the in-memory graph from the Mongo doc + checkpoint when a step starts (cheap — small). <i>Implementation note: in the mock-first build the graph is process-local and kept OUT of the LangGraph checkpoint (the checkpointed state holds only the serialisable record trail), so an in-process resume restores the record trail + lock but not the graph. Cross-process graph rebuild from the durable Mongo doc lands with the real stores (P9); until then the resume guarantee covers the record trail + lock.</i>
Fold (the adapter)	each capability result is mapped into nodes/facts/edges by a per-capability fold-adapter (topology → nodes + depends_on ; metrics → facts on a node; an alert source → an alert node + observed_on). The adapter is the only place a source's shape meets the graph schema (Part D) — a new source = a new fold-adapter, no engine change.
Mutate	folds + the agent's annotate apply to the in-memory graph, validated against the Part D node/edge types.
Checkpoint	after the step, the graph delta is checkpointed (LangGraph) + materialized to the Mongo doc — the read-model the UI + reads use.

B9.5 · Concurrency & reuse

Concurrency: single writer per session (the run-owner lock, B8.2) → no concurrent mutation → no merge conflict. Reads (the UI Graph pane) hit the Mongo read-model from any server.

Reuse across incidents: the graph is per-incident; cross-incident knowledge comes via **related/similar** (the learning loop — the suggestions field), *not* a shared graph. A persistent estate graph, if ever added, is a **read-mostly source** the engine pulls topology from — never the mutable investigation surface.

B9.6 · Edge cases

Case	Handling
Unknown node id	a tool references an id not in the graph → return {id, status: "unknown"} ; the agent adds it (fold/annotate) or picks another path. Never silently invent.
Conflicting facts	two sources disagree on a node's health → keep both (source + confidence + observed_at); the disagreement stays visible; reconcile by confidence/recency. The graph never silently overwrites.
Graph grows large	the render-slice collapses aggressively (path + frontier only); the agent expands on demand; in-memory stays small (per-incident).
Expand too wide	neighbours / walk cap depth + breadth → "N more, collapsed"; the agent narrows.

Cycles (depends_on loops)	traversal is cycle-safe (visited set; networkx handles it).
Orphan node (an alert not yet linked)	allowed; the agent links it (annotate) once it learns the relation.
Idempotent fold (crash re-run)	folding is idempotent — keyed by (node id, fact key, source, observed_at) — so a replayed step never duplicates.
Annotate hallucination	annotate is validated vs the Part D type system and requires an evidence_ref — a finding with no evidence is rejected; bad annotations are auditable + reversible.

B9.7 · Tech stack



C · Capability layer & governance

c1 Connection priority

The engine calls abstract capabilities; the layer binds each source by its **kind**, in this onboarding priority:



Kind	When · examples
native MCP	the tool already speaks MCP → use directly.
dynamic MCP (API)	a REST tool with an OpenAPI spec → register as a dynamic MCP server, no per-endpoint code.
A2A agent	per-layer specialists — App / DB / Network / Storage root-cause; Amelia / BladeLogic wrapped as governed capabilities.

c2 The registry

Entity	Role · key fields
Provider	a source — id · name · kind (skill mcp_local mcp_remote a2a_agent api) · connection · trusted · status · last_synced .
DeclaredCapability	one action — id (provider__action) · provider · description · input_schema · output_schema · effect_hint · intents[] (which needs it satisfies).

CapabilityPolicy

human-owned — `capability_id · effect · access (allow|ask|deny) · status (active|pending_review) · reviewed_by · reviewed_at`. A new capability (registered with no policy) lands `pending_review · deny`, preventing any execution until a human reviews and approves; if a policy row exists (even `pending_review`), `govern()` respects that `access` decision in preference to `DEFAULT`.

DDL is in E1. Declarations are machine-synced and refresh; the policy is human-owned and sticky across syncs.

c3 `govern()` — resolved at call time, every time

Hard deny on missing capability: if the capability is not in the registry, `govern()` returns `deny` immediately and never invokes — a missing/unregistered capability is never allowed to run.

```
if cap not in registry: return Decision(cap_id, unknown, deny, "not in registry")
effect = policy(cap).effect ?? cap.effect_hint ?? "unknown"
access = policy(cap).access ?? (playbook.unknown_access if effect=="unknown"
                                else DEFAULT[trust(cap)][effect])
```

access	behaviour
allow	invoke directly, then audit.
ask	pause (interrupt) → operator approve / refine / deny → on approve, invoke; record the decision.
deny	refuse; the model finds another path or escalates.

c4 Intent resolution

```
def _effect_of(cap, registry):                                # the AUTHORITATIVE effect govern() will use
    pol = registry.policy(cap.id)
    return (pol.effect if pol else None) or cap.effect_hint or Effect.unknown

def resolve_intent(need, phase_effect, registry):
    return rank([c for c in declared_capabilities
                  if need in c.intents                        # satisfies the intent
                  and compatible(_effect_of(c, registry), phase_effect)]) # POLICY-CORRECTED effect, n
```

The playbook names `needs: [metrics, logs, ...]`; resolution turns each into the concrete, allowed capability. **The effect boundary is enforced here at POLICY-CORRECTED effect** — the resolver mirrors `govern()`'s effect calculation, so if a policy overrides a mistyped hint, the resolver uses the corrected effect. A read-only phase thus provably cannot select a write, even if a hint was wrong.

c5 Autonomy ladder & feedback

Operator **feedback** (D5), kept separate from the run, drives an **autonomy ladder** per *phase × node-type × severity*: a proven low-risk action can move ask → allow (earned autonomy); a declared failure / correction moves it back. Autonomy is narrow, evidence-based, and always reversible — the same `CapabilityPolicy` row is what changes. *Implementation note: the autonomy unit is the whole capability row, so an earned allow currently applies engine-wide; finer per-phase × node-type × severity scoping (re-keying the policy + the govern/resolver lookups) is a tracked refinement, not yet built.*

D · The data model

Two schemas. The **Investigation Graph** — the per-incident world (Node · Fact · Edge), shared by all phases. The **Phase Record** — the per-phase journey (PhaseRecord · Step) + a typed output. Every field is here; **NEW** marks a field the phase-output sufficiency check added.

D1 The investigation graph

The incident & cluster rule. A graph has **exactly one incident node** — the subject of *this* investigation. **Other incidents are referenced and actively used** (correlate · carry knowledge · link), never loaded as full nodes — they live in their own documents; `related_to` holds an id + kind. When incidents share a cause, the engine surfaces the match and the **operator** may **link** them into a **cluster** (`member_of`) — a human-confirmed grouping, **not a separate run**; each incident keeps its own session, phases, and closure. One playbook (incident); no "cluster playbook."

Node — a thing in the investigation

```
{
  "id":      "app:payments-api",           // kind:name - stable; everything refers to a node by id
  "kind":    "system",                    // value from the playbook's node_types
  "type":    "app",                       // a type within the kind (or "generic")
  "layer":   "app",                       // system only: business|app|database|network|storage|compute|l
  "name":    "Payments API",
  "labels":  ["pci","tier-0","suspect"],  // open tags; some drive triage
  "props":   { "owner_team":"payments-sre", "on_call":"j.rivera" },
  "facts":   [ /* Fact[] - observations made during the run */ ],
  "summary": "degraded since 14:02; DB-backed latency", "sources": ["servicenow-cmdb","appd"]
}
```

Field	Meaning
id · kind · type · layer	stable id (kind:name); the top category + specific type from the playbook's node_types ; layer (system nodes) tells the engine which deep-dive applies.
name · labels[] · props{}	label; open tags (conventional: suspect · policy_block · central); source attrs + ownership.

facts[] · summary · sources[]	embedded observations (below); rolling summary; tools that contributed.
--	---

Node types — incident-triage's vocabulary

kind	type	represents
system	business_service · app · service · component · database · network_device · storage · compute · zone · region · external_service · generic	the estate, by layer; <code>generic</code> = the escape hatch (never dropped).
incident	incident · cluster	the one subject; or the cluster it belongs to.
change	change	a deploy / config / flag / ticket.
alert	alert	a fired alert — its own node (identity + lifecycle).

Fact — an observation on a node

```
{
  "key": "health", "value": "degraded", "source": "appd",
  "evidence_ref": "appd://payments-api/bt?t=14:02",    // deep link - auditable
  "observed_at": "2026-06-18T14:02Z",                // NO ttl - freshness judged at read time
  "confidence": 0.9,
  "impact_state": "degraded"    // erroring|degraded|stalled|lagging|wrong-data|ok - SEPARATE from health
}
```

Field	Meaning
key · value · source	what's observed + value + the tool.
evidence_ref	deep link to backing data — the fact is auditable (this is why no separate Evidence entity is needed).
observed_at · confidence	no ttl — freshness judged at read time; 0–1, discounted for noisy sources.
impact_state NEW	distinct from <code>health</code> — a <code>health:ok</code> node can still be impacted (lagging, wrong-data) and counts toward blast radius.

Edge — a typed relationship

```
{
  "type": "suspected_cause", "from": "INC-4821", "to": "stor:pay-vol",
  "props": { "confidence":{"value":0.9,"basis":"evidence-complete + matches INC-4820"},
             "rank":1, "path":["app:payments-api","db:payments-ora","stor:pay-vol"]} }
}
```

edge	connects · meaning
part_of · depends_on · runs_on · hosted_on · routes_to · located_in	containment · logical dependency · deployment · storage backing · network path · physical locality.
changed · affects · suspected_cause	a change touched a node · the incident impacts a node · the ranked cause (props: <code>confidence{value,basis}</code> · <code>rank</code> · <code>path</code>).
related_to · member_of	links other incidents (<code>kind</code> = <code>child</code> <code>related</code> <code>caused_by</code> <code>chained</code> <code>similar</code>) · is linked into a cluster (operator-confirmed).
observed_on · correlated_in to	an alert fired on a node · the alert was grouped into the incident/cluster.

Conventions & clustering

rule	meaning
policy_block	a node healthy on its own signals but the cause (a bad firewall rule, expired cert, bad data) — a change/fact on a healthy node on the victim's path → suspect despite health.
realized_by	<code>depends_on.props.realized_by</code> names the physical path of a logical dependency — blast-radius and fault-localization reconcile.
alert_noise · cluster	noisy alerts count for clustering, discounted for cause-confidence; on a shared fingerprint + converging walk the engine surfaces the match and the operator links the incidents into a cluster — never auto, never a separate run.

D2

The phase record

PhaseRecord — the per-phase journey

```
{
  "id": "INC-4821:root-cause:1",    // subject.id:phase:attempt
  "subject": { "domain":"app-incident", "id":"INC-4821", "kind":"incident" }, // SubjectRef
  "phase": "root-cause", "goal": "Find the most probable cause – ranked, with evidence.",
  "state": "done",                // active|waiting_input|waiting_approval|blocked|done|failed
  "plan": "Walk below the app; correlate vs rev47; rank with a path.",
  "steps": [ /* Step[] */ ], "output": { /* the typed output */ },
  "summary": "Cause = pay-vol disk failure, conf 0.9.", "opened_at":"14:04", "closed_at":"14:07"
}
```

Field	Meaning
-------	---------

id · subject	subject.id:phase:attempt (attempt enables backtrack re-runs); the SubjectRef {domain, id, kind} — the one incident this record belongs to (no "cluster scope"); never a bare incident_id.
phase · goal · state · plan	which phase + its goal (from the playbook); lifecycle; what it set out to do.
steps[] · output · summary	the ordered journey (below) — the audit; the typed result (D3); rolling summary.

Step — one entry in the journey

```
{
  "seq":2, "at":"14:04:30", "kind":"tool_call",    // tool_call|reasoning|suggestion|decision|user_input
  "capability":"traces", "input":{"trace":"checkout"}, "result":{"db_span_ms":3140},
  "touched":["svc:checkout","db:payments-ora"],    // → graph node/fact ids
  "evidence":["otel://trace/9af3"], "note":"75% of latency is the DB call"
}
// other kinds: suggestion → {proposal, rationale} · decision → {decision, actor} · user_input → {message}
```

Field	Meaning (by kind)
seq · at · kind	(all) order · timestamp · the discriminator. One ordered log = the audit (subsumes the HLD's Action+HumanInteraction+RunEvent).
capability · input · result	tool_call — the capability (from a need , never hardcoded) + args + return.
touched[] · evidence[]	tool_call — the two hooks: touched → the graph; evidence → the output.
proposal/rationale · decision/actor · message	suggestion · decision (the gate) · user_input.

Worked example — a Root-cause PhaseRecord, connected

```
{ "id":"INC-4821:root-cause:1",
  "subject":{"domain":"app-incident","id":"INC-4821","kind":"incident"}, "phase":"root-cause", "state":"active",
  "steps":[
    {"seq":2,"kind":"tool_call","capability":"traces","result":{"db_span_ms":3140,"pct":"75%"},
      "touched":["svc:checkout","db:payments-ora"],"evidence":["otel://trace/9af3"]},
    {"seq":4,"kind":"tool_call","capability":"metrics(storage)","result":{"disk":"reconstructing since 14:04:30"},
      "touched":["stor:pay-vol"],"evidence":["netapp://aggr01/disk/1.4.7"]},           // ROOT CAUSE
    {"seq":7,"kind":"suggestion","proposal":"Cause: pay-vol disk failure. Conf 0.9."},
    {"seq":8,"kind":"decision","decision":"approve","actor":"j.rivera"} ],           // the gate
  "output":{" candidates":[{"cause":"pay-vol disk failure → RAID rebuild → DB I/O","node":"stor:pay-vol",
    "confidence":{"value":0.9,"basis":"evidence-complete + matches INC-4820"},"rank":1,
    "path":["app:payments-api","db:payments-ora","stor:pay-vol"],
    "evidence":["otel://trace/9af3","netapp://aggr01/disk/1.4.7"]}], "selected":0, "status":"confidence:0.9"}}
```

```
// chain: step.touched → graph · step.evidence → output.candidates[].evidence · suggestion(7) → decision
// graph side-effect on commit: stor:pay-vol gains label `suspect` + an INC-4821 -suspected_cause→ stor:pay-vol
```

D3 The four phase outputs

AssessResult — the framed situation

```
{ "incident_type":"performance", "symptom":"checkout p99 0.4s→4.2s, errors 18%",
  "affected":["app:payments-api","biz:checkout-journey"],
  "impact_assessment":{"scope":"customer-facing","blast_radius":["app:payments-api"],"bounded_by":null,
    "severity":"P1","urgency":"high","business_impact":"checkout / revenue path"},
  "changed":["chg:deploy-rev47"], // [] = checked & clean (not "unknown")
  "time_factor":null, "suspected_locus":{"node":"db:payments-ora","why":"DB span dominates"},
  "related":["INC-4820"], "cluster":null,
  "suggestions":[{"possible_fix":"failover DB to standby","basis":"similar_incident","confidence":0.7},
  "owner":"payments-sre" }
```

Fields: incident_type NEW · symptom NEW · affected[] · impact_assessment (+ bounded_by) · changed[]
 ([] = checked & clean) · time_factor? · suspected_locus? NEW · related[] / cluster? NEW · suggestions[]
 (from priors — the learning loop) · owner .

RootCauseResult — the ranked cause

```
{ "candidates":[{"cause":"","node":"stor:pay-vol",
  "confidence":{"value":0.9,"basis":"evidence-complete + matches INC-4820"}, // basis = WHY
  "rank":1, "path":[...], "evidence":[...], "recommended_fix":"failover DB to standby" }],
  "selected":0, "ruled_out":[{"hyp":"rev47 deploy","evidence":"trace shows DB cost, not app code"}],
  "status":"confident" } // confident | needs-input | escalated
```

Fields: candidates[] (cause · node · confidence{value,basis} NEW · rank · path · evidence · recommended_fix?
 NEW) · selected NEW · ruled_out[] (+ killing evidence) · status .

RemediationResult — the gated fix

```
{ "actions":[{"action_id":"a1", "kind":"mitigate", "technique":"failover", "target":"db:payments-ora",
  "expected_effect":"DB served by standby; I/O normal", "blast_radius":"payments read path",
  "rollback":"fail back once disk replaced", "temporary":true, "revert_when":"incident_close",
  "idempotency_key":"INC-4821-a1", "gated":true,
  "approval":{"decision":"approve","actor":"j.rivera","at":"14:21"},
  "result":"applied; I/O 28ms→4ms", "status":"done" }],
  "followups":[{"detail":"replace disk 1.4.7","basis":"escalate","owner":"storage"}], "status":"applied"
```

Fields: actions[] (kind reverse|mitigate|escalate · technique? · target · expected_effect NEW · blast_radius
 NEW · rollback NEW · temporary? / revert_when? · idempotency_key · gated · approval NEW · result NEW ·
 status) · followups[] · status .

VerifyResult — recovery, confirmed

```
{ "recovered":true, "before_after":"p99 4.2s→260ms, errors 18%→0.2%",
  "watch_window":"15m", "recovery_confidence":0.95,
  "resolution":"failover to standby; durable fix = disk replacement", // what the next incident learn
  "temporary_actions_status":[{"action_id":"a1","status":"scheduled"}], // reverted | scheduled — mu
  "residual":[{"item":"replace disk","owner":"storage"}, "closed_by":"j.rivera", "status":"closed" }
```

Fields: recovered (user's side) · before_after **NEW** · watch_window? / recovery_confidence? **NEW** · resolution **NEW** (the learning-loop field) · temporary_actions_status[] (must be clear) · residual[] · closed_by / status .

D4 Subject & storage planes

```
{ "domain":"app-incident", "id":"INC-4821", "kind":"incident" } // SubjectRef – unique key (domain, id)
```

Plane · store	Holds · shape
data · MongoDB	the incident document — graph + phase outputs + feedback; denormalized , one per subject; one read (D6).
config · PostgreSQL	the playbook (md + version + fields) + the capability registry ; normalized , versioned, governed (E1).
run · LangGraph	the live checkpoint + conversation; the engine's mechanism — not hand-modeled ; materialized into the incident doc on close.

What we purge: no bespoke run-state schema (no Session/Run/PhaseRun/Action/RunEvent/Checkpoint tables) — LangGraph's checkpointer owns the live run; the durable journey is the Phase Record, written into the incident document on close. Planes link by one field: `playbook@version` on the incident, `subject {domain,id}` from the run.

D5 Feedback · D6 Incident document

Feedback — the operator's verdict (separate from the run)

```
{ "_id":"fb-9912", "subject":{"domain":"app-incident","id":"INC-4821"}, "run_id":"INC-4821:run:1",
  "actor":"j.rivera", "kind":"outcome", // outcome | failure | correction
  "verdict":"fix held; good call on the failover", "at":"2026-06-19T09:00Z" }
```

Kept apart from the run so it can't taint the audit; feeds the autonomy ladder (failure/correction lower autonomy) + the suggestions (outcome confirms the resolution).

The incident document — MongoDB (one denormalized doc per incident)

```
// db.incidents – one per subject; nodes conform to the loaded playbook's graph_schema
{ "_id":"INC-4821", "domain":"app-incident", // global subject identity = (domain, _id)
  "incident_type":"performance", "symptom":"checkout p99 0.4s→4.2s, errors 0.5%→18%",
  "priority":"P1", "state":"closed", "owner_team":"payments-sre", "playbook":"incident-triage@1.0.0",
  "impact_assessment":{" ... },
  "graph":{"
    "nodes":[
      {"id":"INC-4821","kind":"incident","type":"incident"}, // the ONE subject node
      {"id":"app:payments-api","kind":"system","type":"app","layer":"app","facts":{"key":"health","va
      {"id":"db:payments-ora","kind":"system","type":"database","layer":"database"},
      {"id":"stor:pay-vol","kind":"system","type":"storage","layer":"storage","labels":["suspect"],"fa
      {"id":"chg:deploy-rev47","kind":"change","type":"change"}, {"id":"alert:7741","kind":"alert","ty
    "edges":[
      {"type":"depends_on","from":"app:payments-api","to":"db:payments-ora"},
      {"type":"hosted_on","from":"db:payments-ora","to":"stor:pay-vol"},
    ]
  }
```

```

{"type":"suspected_cause","from":"INC-4821","to":"stor:pay-vol","props":{"confidence":{"value":0
{"type":"related_to","from":"INC-4821","to":"INC-4820","props":{"kind":"similar"}} ] }, // oth
"phases":[ {"phase":"root-cause","output":{"...}}, {"phase":"remediation","output":{"...}}, {"phase":"veri

```

Indexes: `_id` · `incident_type` · `graph.nodes.id` · `graph.edges.type` · `related_to` + a fingerprint/embedding index for similar-search. Links: `playbook@version` → config; `{domain,id}` ← the run, written through on close.

E · Physical / LLD

Tech stack. Backend **Python** · frontend **React** (single-page) · the engine on **LangGraph** · the in-memory graph on **networkx** (B9) · the data plane **MongoDB** · the config + run-state plane **PostgreSQL** (LangGraph `PostgresSaver`) · live multi-user sync over an append-only event log — **polling** for correctness, **SSE** optional for liveness; **no Redis / WebSocket push bus** (B8) · capabilities reach sources over **MCP** · **dynamic-MCP** · **A2A** (Part C).

E1 PostgreSQL — config plane (normalized)

```

CREATE TABLE playbook (
    id text, version text, domain text, status text, owner text, body_md text,
    phases jsonb, schemas jsonb, defaults jsonb, unknown_access text, changelog text,
    PRIMARY KEY (id, version) );
CREATE TABLE provider (
    id text PRIMARY KEY, name text, kind text,
    connection jsonb, trusted bool, status text, last_synced timestampz );
CREATE TABLE declared_capability (
    id text PRIMARY KEY, provider text REFERENCES provider(id), -- id = provider__action
    description text, input_schema jsonb, output_schema jsonb,
    effect_hint text, intents text[], last_synced timestampz ); -- intents[] = which needs it satisfi
CREATE TABLE capability_policy (
    capability_id text PRIMARY KEY REFERENCES declared_capability(id),
    effect text, access text, status text,
    reviewed_by text, reviewed_at timestampz );

```

The LangGraph checkpointer uses its own Postgres tables (`PostgresSaver`); `thread_id` = the run id. We do not hand-model them (D4).

E2 MongoDB — data plane (denormalized)

```

// db.incidents — one document per subject (shape = D6)
{ _id, domain, incident_type, symptom, priority, state, owner_team, playbook,
  opened_at, closed_at, impact_assessment, graph:{nodes,edges}, phases:[{phase,output}] }
// db.feedback — operator verdicts, separate from the run (D5)
{ _id, subject:{domain,id}, run_id?, actor, kind, verdict, note, at }
// indexes
db.incidents.createIndex({ incident_type:1, state:1 })
db.incidents.createIndex({ "graph.nodes.id":1 }); db.incidents.createIndex({ "graph.edges.type":1 })
db.incidents.createIndex({ fingerprint:1 }) // + a vector index for similar-incident search

```

E3 Engine API (surface)

The endpoint contract. Each endpoint returns status + a consistent shape. Membership / roles re-checked per request (AC9 — a revoke drops the operator immediately). `actor` is a required query param on all reads; writes route through `require_writer` (pen-holder only). The run-owner lock serializes `/advance` and `/gate → resume` so only one server drives the thread.

Endpoint	Contract	Returns
POST /sessions	<code>{domain, id, kind, actor}</code> → create or join a session; no run started yet; idempotent on subject. Checks related incidents + asks the operator to link or open new.	<code>session_id</code> , <code>status</code> , <code>members[]</code> , <code>pen_holder</code> , <code>role</code>
POST /sessions/{sid}/advance	acquire run-owner lock; if not started, start the run (seed graph → begin Assess); drain queued input; step to completion or gate pause. Release lock. 409 if paused at a gate (use <code>/gate</code>) or locked by another worker.	<code>session_id</code> , <code>status</code> (<code>waiting_approval done</code>), <code>next</code> , <code>pen_holder</code>
POST /sessions/{sid}/gate	<code>{actor, gate_id, decision}</code> ; writer-only (403 if not pen-holder). <code>gate_id</code> must match engine's pending pause, else 409. On <code>approve</code> : acquire lock → resume → fold decision Step → release lock. On <code>refine</code> : gate stays open (no lock). On <code>deny</code> : record denial, set read-model terminal state, halt.	<code>session_id</code> , <code>status</code> (<code>waiting_approval denied done</code>), <code>gate{gate_id, decision, actor}</code> , <code>next</code>
GET /sessions/{sid}/incident	<code>?actor=</code> ; re-check membership (403). Return the durable read-model from Mongo (graph + phase outputs + terminal state).	incident document (D6) or 404 if not yet written
POST /sessions/{sid}/messages	<code>{actor, text}</code> ; writer-only (403). Enqueue into the session's input queue (drained on next <code>/advance</code>). Append event to the one ordered stream.	<code>seq</code> (the event sequence number)
POST /sessions/{sid}/take-pen	<code>{actor}</code> ; must be a member (403). Release from the current holder; grant to <code>actor</code> . (One pen-holder at a time.)	<code>ok</code> , <code>pen_holder</code> , <code>role</code> (<code>writer viewer</code>)
POST /sessions/{sid}/release-pen	<code>{actor}</code> ; must be the current pen-holder. Release; no new holder until take-pen.	<code>ok</code> , <code>pen_holder</code>
GET /sessions/{sid}/events	<code>?actor=</code> , <code>?after_seq=0</code> ; re-check membership (403). Return events appended since <code>after_seq</code> .	<code>events[]</code> (ordered by <code>seq</code>)
GET /sessions/{sid}/stream	<code>?actor=</code> , <code>?after_seq=0</code> ; SSE over text/event-stream. Resume from <code>Last-Event-ID</code> header or <code>after_seq</code> ; retry via browser EventSource. Same <code>seq</code> as <code>/events</code> and <code>/poll</code> .	<code>text/event-stream</code> ; each line: <code>id: {seq}\ndata: {json}\n\n</code>

GET /sessions/{sid}/poll	?actor=, ?after_seq=0 ; re-check membership (403). Return snapshot (read-model + status + pen_holder + role) + new events since seq. Single stateless call — no push infra needed.	incident, events[], seq, status (waiting_approval denied done new), pen_holder, role
POST /feedback	{domain, id, kind, actor, verdict?, note?, run_id?} ; kind = outcome failure correction (feeds autonomy ladder + learning loop).	stored, kind
POST /playbooks · /providers · /policies	config-plane admin (author/validate/publish a playbook; register a provider; set a policy).	varies

E4 Errors, retries, recovery

In the phase loop (B3): a capability invocation raises `TransientError` or `PermanentError` (B4 classify, not the capability — the layer wraps raw exceptions). Transient → **retry per defaults.retry** (max attempts; no backoff in-process); permanent → **record the error Step** (kind=tool_call, result={error: ...}) → **check on_failure** :

on_failure mode	Behaviour	
run-remaining	finish any remaining independent steps in the phase; then set state=blocked (partial) and close. The run continues to the next phase ; the escalation lives in the audit (the error Step is recorded; no separate error_handler fires).	
default (halt)	fire the playbook's error_handler immediately	append a reasoning Step (result={escalated: true, error: ...}) with the handler's action/to/via, set state=failed , close the phase, halt the run (do not advance). The error-handler is an escalate Step — a persistent audit event, not config code that runs.

Exactly-once & checkpointing: every write carries an `idempotency_key` ; on resume after a crash, the layer checks *seen* and returns the cached result instead of re-invoking. The run state (graph + messages + phase records) is checkpointed after every step (LangGraph), so a second server resumes from the last checkpoint, re-runs any in-flight step (reads idempotent, writes guarded by idempotency_key).

Backtrack: Verify's `recovered` field determines the edge (B6). If false, the run backtracks to Root cause as a new attempt (same session, new `phase_records` entry with attempt++); the

graph + prior outputs stay (context for the retry). A halted or denied run ends (status=failed or denied); human closes the incident.

F · Operations

F1 The console

The only operator UI — a **React** single-page app, in the Claude-Desktop three-pane shape: **Incidents** (left) · **Triage** — the whole incident's chat with the **approval gate inline** (center) · **Phases & Steps** with a **Graph** tab (right). It evolves as autonomy is earned (the autonomy badge).

Live sync & message flow

The UI runs a **polling loop** (or SSE for liveness) to fetch `/poll` every ~2 seconds (with `after_seq` to resume from the last event). Each poll returns: the read-model snapshot (incident + graph + phases) + new events since the client's seq + the current `status` + the `pen_holder`. Messages are typed by the UI user (pen-holder only) and sent via `POST /messages`, enqueued in the session, then drained into the run on the next `/advance`. So a message does NOT block — the operator sends it, sees it appended to the event stream, then a separate `/advance` call drives the run forward. (In future, a single `/message + auto-advance` endpoint may conflate these.)

The gate, inline: when the run pauses (status=waiting_approval), the gate prompt appears in the chat with the proposed action + expected_effect + blast_radius + rollback. The pen-holder clicks `approve | refine | deny`; the UI calls `POST /gate {gate_id, decision, actor}`. On `approve`, the server resumes the run. On `deny`, the run halts with status=denied (terminal). On `refine`, the gate stays open — the operator can send a follow-up message (via `/messages`) asking for a different action; the agent loops and proposes again. The pen is held until explicitly released or the incident closes.

Roles: the session has a `pen_holder` (the only writer) and viewers. The UI renders `/poll`'s `role` field (writer|viewer) to gate the send-message + approve buttons. A revoke (via `require_member` re-check at the server) drops the operator from the next event poll — they lose access to chat + incident doc immediately, without requiring a page refresh.

Investigation

app-incident · INC-4821 — payments-api

autonomy: assisted

ACTIVE INCIDENTS

INC-4821 · payments-api

P1 · Root cause done

INC-4806 · orders-api

P3 · Assess

INC-4821

checkout journey · performance · P1

Symptom **p99 0.4→4.2s · err 18%**

Owner **payments-sre · j.rivera**

Blast **12 svc · ~1.2k users**

Business **revenue path**

Change **rev47 · ruled out**

subject · { app-incident , INC-4821 }

RELATED INCIDENTS

Similar

INC-4820 · DB slow → failover (gave the fix)

Related

INC-4806 · orders-api (downstream)

ASSESS

Triage agent

Checkout **p99 0.4s→4.2s**, errors **18%** — P1, customer-facing (~1.2k users, revenue path). Built the graph: **147 nodes** in scope, **12 impacted**. Deploy **rev47** landed 13:50; a similar prior, **INC-4820**, was fixed by failover.

ServiceNow AppD topology similar · INC-4820

ROOT CAUSE

Triage agent

Not the deploy. Walked the path **app → DB → storage: pay-vol** disk failed **13:58** → RAID rebuild → DB I/O 28ms → checkout latency. Confidence **0.9** — *evidence-complete + matches INC-4820*.

otel trace AWR NetApp 3 ruled out

You

rev47 landed right before — sure it's clean?

Triage agent

Yes — the trace shows **75%** of latency is the DB call, not app code. Onset aligns with the **13:58** disk, not the 13:50 deploy. rev47 ruled out.

Suggested fix — fail over db:payments-ora → standby to stop the bleeding. Durable fix = replace disk 1.4.7. Same play resolved INC-4820.

Approval needed — Remediation (a write · gated)

failover db:payments-ora → standby

expects: DB on standby, I/O normal · blast: payments read path · rollback: fail back once disk replaced · temporary → revert when disk replaced

Approve Refine Deny

Phases & steps Graph

Assess

done

read incident · ServiceNow

build graph · 147 nodes / 12 impacted

changes in window · rev47

link similar · INC-4820

→ AssessResult · perf · P1 · blast 12 svc · 1 suggestion

Root cause

done · 4/4

trace · 75% is the DB call

metrics(db) · I/O 28ms

metrics(storage) · disk reconstructing

rule out · net / compute / rev47

→ RootCauseResult · stor:pay-vol · 0.9 [+3 ruled out]

Remediation

awaiting approval

failover db:payments-ora → standby

gated · expected_effect · blast · rollback

Verify & close

queued

synthetic replay · before/after

revert temporary · record resolution

EVERY STEP → TOOL CALL + EVIDENCE, LOGGED

Right pane on **Phases & Steps** — the four phases, each step, and its typed output (worked run: INC-4821).

Investigation

app-incident · INC-4821 — payments-api

autonomy: assisted

ACTIVE INCIDENTS

INC-4821 · payments-api

P1 · Root cause done

INC-4806 · orders-api

P3 · Assess

INC-4821

checkout journey · performance · P1

Symptom **p99 0.4→4.2s · err 18%**

Owner **payments-sre · j.rivera**

Blast **12 svc · ~1.2k users**

Business **revenue path**

Change **rev47 · ruled out**

subject · { app-incident , INC-4821 }

RELATED INCIDENTS

Similar

INC-4820 · DB slow → failover (gave the fix)

Related

INC-4806 · orders-api (downstream)

ASSESS

Triage agent

Checkout **p99 0.4s→4.2s**, errors **18%** — P1, customer-facing (~1.2k users, revenue path). Built the graph: **147 nodes** in scope, **12 impacted**. Deploy **rev47** landed 13:50; a similar prior, **INC-4820**, was fixed by failover.

ServiceNow AppD topology similar · INC-4820

ROOT CAUSE

Triage agent

Not the deploy. Walked the path **app → DB → storage: pay-vol** disk failed **13:58** → RAID rebuild → DB I/O 28ms → checkout latency. Confidence **0.9** — *evidence-complete + matches INC-4820*.

otel trace AWR NetApp 3 ruled out

You

rev47 landed right before — sure it's clean?

Triage agent

Yes — the trace shows **75%** of latency is the DB call, not app code. Onset aligns with the **13:58** disk, not the 13:50 deploy. rev47 ruled out.

Suggested fix — fail over db:payments-ora → standby to stop the bleeding. Durable fix = replace disk 1.4.7. Same play resolved INC-4820.

Approval needed — Remediation (a write · gated)

failover db:payments-ora → standby

expects: DB on standby, I/O normal · blast: payments read path · rollback: fail back once disk replaced · temporary → revert when disk replaced

Approve Refine Deny

Phases & steps Graph

147 nodes in scope · 12 impacted

cause path · blast · full

FULL TOPOLOGY · 147 nodes — engine built it all; the view focuses focus

INC-4821 incident · subject

12 impacted neighbours affects

cart-svc checkout-ui ledger-svc payments-api app · degraded depends_on payments-ora database hosted_on pay-vol storage SUSPECT · cause 0.9 +135 healthy · collapsed

Focused view — the cause path (3) + the impacted neighbours (12) out of **147**; healthy nodes collapsed. The engine holds the full graph; the console shows the few that matter.

amber bold = the cause path (suspected_cause)

thin amber = impacted neighbours (blast radius)

grey = ruled out · collapsed · healthy

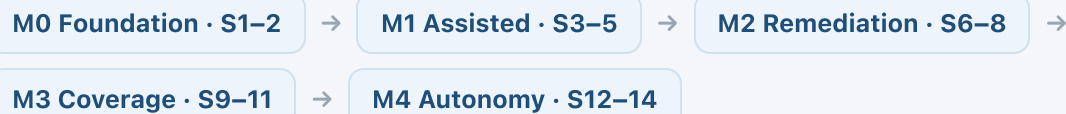
Right pane switched to **Graph** — the investigation graph, cause path **app → db → storage** highlighted to the suspect; the deploy greyed as ruled-out, the similar prior a reference. One incident node (the subject); others are references.

F2 Cross-cutting

Concern	Design
Audit	the Step journey is append-only and IS the audit trail; every tool-call carries an evidence deep-link; approvals record actor + decision + time. Regulator-grade, reconstructable.
Security / hardening	secrets / KMS, encryption at-rest + in-transit, least-privilege capability scopes, audit immutability — in place before any write ships (gates M2).
Safety	read-only phases provably cannot write (the effect boundary is enforced at intent resolution, C4); every write is gated + reversible.
DR / resilience	checkpointed run state (resumable); RTO/RPO targets; a need with no healthy capability degrades gracefully + asks.
Feedback / learning	operator verdicts (D5) feed the autonomy ladder (C5) + the similar-incident suggestions.
Observability	the engine emits its own metrics (phase latency, gate decisions, autonomy mix) for the autonomy dashboard.

G · Delivery

G1 Milestones & timeline



Milestone	Ships · exit
M0 Foundation	engine skeleton, registry, stores, <code>govern()</code> , checkpointer, console shell — an assisted run end-to-end on a read-only capability, in shadow.
M1 Assisted	Assess + Root cause (read-only), the full console + HITL surface — triages real incidents read-only on a pilot slice.
M2 Remediation	the first gated writes + Verify & close + rollback + <code>error_handler</code> — gated fixes execute safely, exactly-once. (Hardening baseline green.)
M3 Coverage	multi-app topology + incident relationships + A2A specialists + Amelia wrapped + playbook studio — handles a multi-app incident; N playbooks live.
M4 Autonomy	earned autonomy for proven low-risk actions + the feedback→policy loop + dashboard; Amelia ramped down per proven slice.

Two parallel tracks: **hardening** gates production writes (in place before M2); **Amelia** runs alongside, is wrapped as a governed A2A capability, and is dialled down per proven slice. The full by-sprint plan + Gantt is the companion **Delivery Plan**.

NFR	Target
Auditability	every action + decision reconstructable from the Step journey; immutable, regulator-grade.
Safety	no un-gated write; every write reversible; read-only phases provably cannot write.
Latency	assisted triage returns a first assessment in minutes; the gate is the only human wait.
Availability / scale	concurrent incidents; degrade gracefully when a source is down; engine horizontal.
Extensibility	a new source needs only a registry entry; a new domain only a new playbook — no engine change.

The complete design, in one place: **A** system · **B** the engine on LangGraph · **C** capability layer · **D** data model · **E** physical/LLD · **F** operations · **G** delivery. Companion: the **Presentation** (leadership) and the **Delivery Plan** (by-sprint Gantt). Governed at the boundary · regulator-grade audit · earned, reversible autonomy · no vendor or domain lock-in.